
Análisis e Implementación del Algoritmo DDA

Rasterización de líneas en sistemas gráficos discretos



Bryan Robert Ninamango Arroyo

Computer Science Student

Descripción

Este artículo presenta el análisis matemático, la formulación teórica y la implementación práctica del algoritmo Digital Differential Analyzer (DDA), una técnica clásica utilizada para la rasterización de líneas en gráficos por computadora. Asimismo, se estudia su complejidad computacional y se valida su funcionamiento mediante una aplicación interactiva desarrollada en Python utilizando el framework Flask.

Contents

1	Introducción	4
2	Abstract	5
3	Fundamento teórico	6
3.1	Gráficos por computadora	6
3.2	Representación de píxeles en una pantalla	6
3.3	Sistema de coordenadas en pantalla	7
3.4	Algoritmos de generación de líneas	8
3.5	Pendiente y función lineal	8
4	Formulación del algoritmo	9
4.1	Cálculo de las diferencias	9
4.2	Determinación del número de pasos	9
4.3	Cálculo de los incrementos	10
4.4	Generación de los puntos de la línea	10
4.5	Conversión a píxeles	10
4.6	Complejidad computacional	10
5	Implementación	11
5.1	Entorno y lenguaje de programación	11
5.2	Pseudocódigo del algoritmo	11
5.3	Código fuente	12
5.4	Descripción del código	12
5.5	Ejemplo de ejecución	13
6	Resultados	13
7	Conclusiones	14
	Referencias	15

List of Figures

1	Utah teapot.	6
2	Estructura de píxeles en una imagen digital.	7
3	Sistema de coordenadas de píxeles.	7
4	Trazado de línea.	8
5	Resultado del trazado de una línea mediante el algoritmo DDA.	14

List of Tables

1	Ejemplo de ejecución DDA.	13
---	-----------------------------------	----

1 Introducción

La representación de figuras geométricas sobre una pantalla digital constituye uno de los problemas fundamentales de los gráficos por computadora. A diferencia del plano matemático continuo, las pantallas están compuestas por una cuadrícula discreta de píxeles, lo que impide representar objetos geométricos de forma exacta y exige el uso de algoritmos de aproximación que determinen qué píxeles deben activarse para reproducir fielmente una figura dada.

Entre los elementos geométricos más básicos se encuentra la línea recta, cuya representación sobre una superficie rasterizada requiere calcular de manera eficiente las posiciones intermedias entre dos puntos extremos. Para resolver este problema se han desarrollado diversos algoritmos de generación de líneas, entre los cuales destaca el algoritmo DDA (*Digital Differential Analyzer*), uno de los primeros enfoques sistemáticos para el trazado de líneas en sistemas gráficos digitales.

El presente artículo tiene como objetivo exponer el fundamento teórico, la formulación matemática y la implementación del algoritmo DDA, con el propósito de comprender su funcionamiento y evaluar su desempeño como herramienta de rasterización. Su elección responde a la facilidad de implementación que ofrece, lo que permitió centrar el desarrollo del trabajo en la formulación matemática que sustenta el algoritmo y en explicar de qué manera los incrementos sucesivos derivados de la ecuación de la recta constituyen una solución concreta al problema de representar líneas sobre una superficie discreta.

Palabras clave: algoritmo DDA, rasterización, gráficos por computadora, trazado de líneas, formulación matemática, Python.

2 Abstract

The representation of geometric figures on a digital screen is one of the fundamental problems in computer graphics. Unlike the continuous mathematical plane, digital displays are composed of a discrete grid of pixels, making it impossible to represent geometric objects exactly and requiring the use of approximation algorithms to determine which pixels must be activated in order to faithfully reproduce a given shape.

Among the most basic geometric primitives is the straight line, whose representation on a rasterized surface requires the efficient computation of intermediate positions between two endpoints. To address this problem, several line-generation algorithms have been developed, among which the Digital Differential Analyzer (DDA) algorithm stands out as one of the earliest systematic approaches for line drawing in digital graphics systems.

This paper aims to present the theoretical foundations, mathematical formulation, and implementation of the DDA algorithm in order to understand its operation and evaluate its performance as a line rasterization technique. Its selection responds to the ease of implementation it offers, which allowed the development of this work to focus on the mathematical formulation underlying the algorithm and on explaining how the successive increments derived from the line equation constitute a concrete solution to the problem of representing lines on a discrete surface.

Keywords: DDA algorithm, rasterization, computer graphics, line drawing, mathematical formulation, digital differential analyzer, Python.

3 Fundamento teórico

3.1 Gráficos por computadora

Los gráficos por computadora constituyen una rama de la informática enfocada en la generación, transformación y visualización de información mediante representaciones gráficas, utilizando algoritmos para crear imágenes a partir de modelos digitales (Hearn & Baker, 2014).

Esta disciplina abarca desde el modelado geométrico de objetos bidimensionales y tridimensionales hasta técnicas de renderizado, iluminación, texturizado y animación, permitiendo simular escenas realistas o estilizadas en pantalla (Shirley & Marschner, 2009). Según Foley et al. (1996), los gráficos por computadora integran fundamentos matemáticos, de hardware y de procesamiento visual para facilitar la interacción entre el usuario y los sistemas digitales, encontrando aplicaciones en áreas como videojuegos, simulación, diseño asistido, visualización científica y realidad virtual.

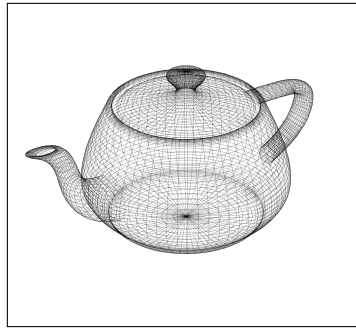


Figure 1: Utah teapot.

3.2 Representación de píxeles en una pantalla

Las imágenes digitales están compuestas por píxeles, unidades mínimas de representación visual organizadas en una cuadrícula bidimensional que, en conjunto, forman la imagen completa (Hearn & Baker, 2014). Debido a esta estructura discreta, los elementos gráficos continuos, como las líneas y las curvas, no pueden representarse de manera exacta sobre una pantalla; en su lugar, deben aproximarse mediante la activación de un conjunto de píxeles distribuidos a lo largo de su trayectoria (Foley et al., 1996).

Este proceso de aproximación constituye la base de los sistemas gráficos rasterizados, en los cuales cada imagen se construye a partir de una matriz de píxeles. La calidad visual de la representación depende de factores como la resolución de la pantalla y la precisión con la que se seleccionan los píxeles que conforman cada figura. En consecuencia, los algoritmos de generación de líneas desempeñan un papel fundamental al determinar qué posiciones de la cuadrícula deben activarse para representar de la forma más fiel posible un segmento geométrico continuo.

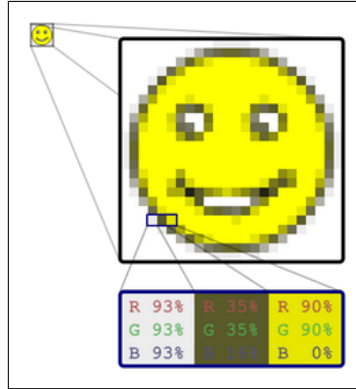


Figure 2: Estructura de píxeles en una imagen digital.

Nota. Adaptado de HandWiki (2024).

3.3 Sistema de coordenadas en pantalla

Para localizar y manipular los píxeles que componen una imagen digital, los sistemas gráficos emplean un sistema de coordenadas bidimensional. Cada píxel se identifica mediante un par de coordenadas enteras (x, y) , donde x indica la posición horizontal e y la posición vertical dentro de la cuadrícula de la pantalla (Hearn & Baker, 2014).

A diferencia del plano cartesiano convencional, el sistema de coordenadas en pantalla sitúa el origen $(0, 0)$ en la esquina superior izquierda. En esta disposición, los valores de x aumentan hacia la derecha, mientras que los valores de y aumentan hacia abajo (Foley et al., 1996). Esta orientación es característica de la mayoría de los sistemas gráficos digitales modernos y permite establecer una correspondencia directa entre las coordenadas y las posiciones físicas de los píxeles en la pantalla.

La localización precisa de los píxeles mediante coordenadas constituye un aspecto fundamental en los gráficos por computadora, ya que los algoritmos de rasterización utilizan esta información para representar objetos geométricos sobre una superficie discreta. En particular, los algoritmos de generación de líneas emplean las coordenadas de los puntos inicial y final de un segmento para calcular las posiciones intermedias que deben activarse durante el proceso de trazado (Shirley & Marschner, 2009).

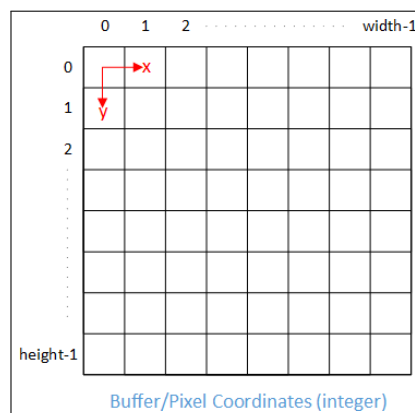


Figure 3: Sistema de coordenadas de píxeles.

Nota. Adaptado de Eurosys (s.f).

3.4 Algoritmos de generación de líneas

El trazado de líneas sobre una pantalla de píxeles presenta un desafío fundamental: una línea recta es un objeto geométrico continuo, mientras que la pantalla está compuesta por una cuadrícula discreta de píxeles (Foley et al., 1996). Por ello, no es posible representar una línea de forma exacta; en su lugar, es necesario seleccionar el conjunto de píxeles que mejor aproxime su trayectoria visual.

La selección de estos píxeles debe cumplir dos objetivos principales: precisión, para que la línea representada sea lo más fiel posible a la línea geométrica ideal, y eficiencia, para que el cálculo de las posiciones se realice con el menor costo computacional posible (Hearn & Baker, 2014). Estas características son especialmente importantes en aplicaciones gráficas que requieren procesamiento en tiempo real.

Con el propósito de resolver este problema, se han desarrollado diversos algoritmos de generación de líneas. Entre los más conocidos se encuentran el algoritmo DDA (*Digital Differential Analyzer*) y el algoritmo de Bresenham, los cuales emplean diferentes estrategias para determinar los píxeles que conforman una línea sobre una superficie rasterizada (Shirley & Marschner, 2009). En particular, el algoritmo DDA utiliza incrementos sucesivos basados en la pendiente de la recta para calcular las posiciones intermedias necesarias para su representación.

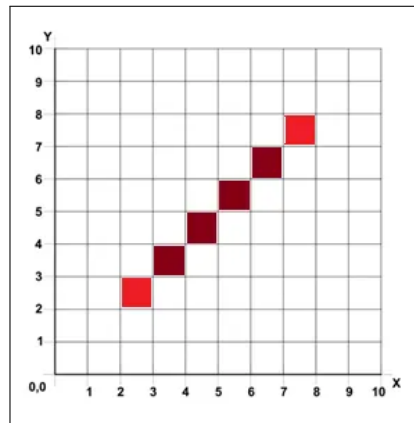


Figure 4: Trazado de línea.

Nota. Adaptado de Medium (s.f).

3.5 Pendiente y función lineal

La pendiente es un concepto fundamental de la geometría analítica que describe la inclinación de una recta respecto al eje horizontal. Matemáticamente, se define como la razón entre la variación vertical Δy y la variación horizontal Δx entre dos puntos (x_1, y_1) y (x_2, y_2) (Hearn & Baker, 2014):

$$m = \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1} \quad (1)$$

La función lineal que describe una recta se expresa mediante la ecuación:

$$y = mx + b \quad (2)$$

donde m representa la pendiente y b el intercepto con el eje y . A partir de esta ecuación, dado un valor de x , es posible calcular el valor correspondiente de y y, por tanto, determinar distintos puntos pertenecientes a la recta (Foley et al., 1996).

Asimismo, la ecuación lineal permite generar puntos sucesivos a lo largo de la trayectoria mediante incrementos constantes en las coordenadas. Cuando se avanza una unidad en la dirección horizontal, el valor de y varía en una cantidad equivalente a la pendiente de la recta:

$$x_{i+1} = x_i + 1, \quad y_{i+1} = y_i + m \quad (3)$$

Este principio constituye la base para la generación incremental de puntos sobre una línea. En los gráficos por computadora, dichas coordenadas deben adaptarse a la naturaleza discreta de la pantalla, seleccionando los píxeles que mejor aproximen la trayectoria geométrica ideal (Hearn & Baker, 2014).

La relación entre la pendiente y la función lineal constituye el fundamento matemático del algoritmo DDA (*Digital Differential Analyzer*), el cual utiliza incrementos sucesivos en las coordenadas para determinar de manera eficiente los píxeles que deben activarse durante el trazado de una línea (Shirley & Marschner, 2009).

4 Formulación del algoritmo

El algoritmo DDA (*Digital Differential Analyzer*) se basa en la generación incremental de puntos a lo largo de una recta, aprovechando la relación lineal entre las coordenadas x e y definida por la pendiente (Hearn & Baker, 2014). A partir de las diferencias entre los puntos extremos del segmento, el algoritmo calcula incrementos constantes que permiten obtener de manera eficiente las coordenadas intermedias de la línea.

4.1 Cálculo de las diferencias

El primer paso consiste en calcular las diferencias entre los extremos del segmento (Foley et al., 1996):

$$\Delta x = x_2 - x_1, \quad \Delta y = y_2 - y_1 \quad (4)$$

donde Δx representa el desplazamiento horizontal y Δy el desplazamiento vertical entre el punto inicial (x_1, y_1) y el punto final (x_2, y_2) .

4.2 Determinación del número de pasos

El número de iteraciones que realizará el algoritmo se determina tomando la mayor variación entre $|\Delta x|$ y $|\Delta y|$ (Shirley & Marschner, 2009):

$$steps = \max(|\Delta x|, |\Delta y|) \quad (5)$$

Este criterio garantiza que el avance entre puntos consecutivos sea uniforme y que la línea se represente de forma continua sobre la cuadrícula de píxeles.

4.3 Cálculo de los incrementos

Una vez determinado el número de pasos, se calculan los incrementos que se aplicarán a las coordenadas en cada iteración (Hearn & Baker, 2014):

$$x_{inc} = \frac{\Delta x}{steps}, \quad y_{inc} = \frac{\Delta y}{steps} \quad (6)$$

Estos valores representan el avance proporcional en cada eje por cada paso del algoritmo, permitiendo recorrer el segmento desde el punto inicial hasta el punto final mediante incrementos constantes.

4.4 Generación de los puntos de la línea

El algoritmo comienza en el punto inicial (x_1, y_1) y genera los puntos intermedios sumando sucesivamente los incrementos calculados (Foley et al., 1996):

$$x_{k+1} = x_k + x_{inc}, \quad y_{k+1} = y_k + y_{inc} \quad (7)$$

Este proceso se repite hasta completar el número de pasos establecido, obteniendo una secuencia de coordenadas que aproxima la trayectoria de la recta.

4.5 Conversión a píxeles

Debido a que las coordenadas generadas pueden contener valores reales, estas deben convertirse a coordenadas enteras para poder representarse en la pantalla. Para ello, cada punto calculado se aproxima al píxel más cercano mediante una operación de redondeo (Shirley & Marschner, 2009):

$$x_{pixel} = \text{round}(x_k), \quad y_{pixel} = \text{round}(y_k) \quad (8)$$

Finalmente, el píxel ubicado en la posición (x_{pixel}, y_{pixel}) se activa en cada iteración, produciendo la representación discreta de la línea sobre la cuadrícula de la pantalla.

4.6 Complejidad computacional

El rendimiento del algoritmo DDA está determinado por el número de píxeles necesarios para representar el segmento de línea. Dicho valor se obtiene mediante

$$steps = \max(|\Delta x|, |\Delta y|)$$

que corresponde al número de iteraciones requeridas para recorrer la línea desde el punto inicial hasta el punto final.

Una vez calculados los incrementos en los ejes x e y , el algoritmo ejecuta un único ciclo iterativo en el que, para cada paso, se realizan operaciones de costo constante: actualización de coordenadas, redondeo y almacenamiento del píxel generado. Debido a que el número de iteraciones es proporcional a $steps$, la complejidad temporal del algoritmo es

$$T(n) = \mathcal{O}(n),$$

donde $n = \max(|\Delta x|, |\Delta y|)$.

Respecto al uso de memoria, la implementación desarrollada almacena todos los píxeles generados en una lista antes de retornarlos. En consecuencia, el espacio requerido también crece de forma lineal con el número de puntos calculados, por lo que la complejidad espacial es

$$S(n) = \mathcal{O}(n).$$

Aunque el algoritmo presenta una complejidad temporal óptima para este tipo de problema, el empleo de aritmética de punto flotante introduce operaciones de división y redondeo en cada iteración. Por esta razón, algoritmos como el de Bresenham suelen ofrecer un mejor rendimiento práctico al utilizar únicamente aritmética entera, manteniendo la misma complejidad asintótica.

5 Implementación

5.1 Entorno y lenguaje de programación

El algoritmo DDA (Digital Differential Analyzer) fue implementado en el lenguaje de programación Python, seleccionado por su sintaxis clara y su capacidad para representar de manera sencilla las operaciones matemáticas involucradas en el trazado de líneas. La implementación se estructuró mediante una función principal que recibe las coordenadas de los puntos extremos del segmento y genera como salida la secuencia de píxeles que conforman la línea resultante. El código fuente completo del proyecto se encuentra disponible en el siguiente repositorio:

<https://github.com/Bryannsss140101/algorithm-DDA>

5.2 Pseudocódigo del algoritmo

A continuación se muestra el pseudocódigo correspondiente al algoritmo DDA empleado para el trazado de líneas.

Algorithm 1 DDA

Require: Punto inicial $P_0 = (x_1, y_1)$, punto final $P_1 = (x_2, y_2)$

Ensure: Lista de píxeles que conforman la línea

```

1:  $\Delta x \leftarrow x_2 - x_1$ 
2:  $\Delta y \leftarrow y_2 - y_1$ 
3:  $steps \leftarrow \max(|\Delta x|, |\Delta y|)$ 
4: if  $steps = 0$  then
5:   return  $\{(x_1, y_1)\}$ 
6: end if
7:  $x_{inc} \leftarrow \Delta x / steps$ 
8:  $y_{inc} \leftarrow \Delta y / steps$ 
9:  $x \leftarrow x_1, \quad y \leftarrow y_1$ 
10:  $L \leftarrow \emptyset$ 
11: for  $i = 0$  to  $steps$  do
12:   Agregar  $(\text{round}(x), \text{round}(y))$  a  $L$ 
13:    $x \leftarrow x + x_{inc}$ 
14:    $y \leftarrow y + y_{inc}$ 
15: end for
16: return  $L$ 
```

5.3 Código fuente

A continuación se presenta la implementación del algoritmo DDA en el lenguaje de programación Python.

Listing 1 Implementación del algoritmo DDA en Python.

```
1 def DDA(p0: Position, p1: Position) -> list:
2     points = []
3
4     # Cálculo de las diferencias entre los puntos extremos
5     dif_x = p1.x - p0.x
6     dif_y = p1.y - p0.y
7
8     # Determinación del número de pasos
9     steps = max(abs(dif_x), abs(dif_y))
10
11     # Caso especial: puntos coincidentes
12     if steps == 0:
13         points.append(Position(p0.x, p0.y))
14         return points
15
16     # Cálculo de los incrementos en cada eje
17     x_inc = dif_x / steps
18     y_inc = dif_y / steps
19
20     x = p0.x
21     y = p0.y
22
23     # Generación de los píxeles de la línea
24     for _ in range(steps + 1):
25         points.append(Position(round(x), round(y)))
26         x += x_inc
27         y += y_inc
28
29     return points
```

5.4 Descripción del código

La función recibe dos objetos **Position** que representan los puntos extremos del segmento de línea. En primer lugar, se calculan las diferencias **dif_x** y **dif_y** entre las coordenadas de ambos puntos, de acuerdo con la ecuación (4). Posteriormente, el número de pasos, denominado **steps**, se determina como el máximo entre los valores absolutos de dichas diferencias, tal como establece la ecuación (5). Esta elección garantiza un avance uniforme a lo largo de la línea y evita omisiones de píxeles en cualquiera de los ejes.

Asimismo, se considera el caso especial en que **steps** es igual a cero, situación que ocurre cuando los puntos inicial y final coinciden. En este caso, la función retorna directamente dicho punto sin necesidad de ejecutar el proceso iterativo.

Una vez determinado el número de pasos, se calculan los incrementos **x_inc** e **y_inc** mediante la división de las diferencias correspondientes entre **steps**, siguiendo la ecuación (6). Estos incrementos representan la variación que experimentarán las coordenadas en cada iteración.

El bucle principal se ejecuta durante `steps + 1` iteraciones con el fin de incluir tanto el punto inicial como el punto final del segmento. En cada iteración, las coordenadas acumuladas se redondean al entero más cercano mediante la función `round()`, obteniendo así la posición discreta del píxel correspondiente, de acuerdo con la ecuación (8). Finalmente, las coordenadas se actualizan sumando los incrementos calculados, conforme a la ecuación (7), permitiendo avanzar progresivamente desde el punto inicial hasta el punto final y generar la secuencia completa de píxeles que conforman la línea.

5.5 Ejemplo de ejecución

Para ilustrar el funcionamiento del algoritmo, se considera el segmento definido por los puntos $P_0 = (0,0)$ y $P_1 = (5,3)$. Aplicando las ecuaciones descritas anteriormente se obtiene:

$$\Delta x = 5, \quad \Delta y = 3, \quad steps = \max(5, 3) = 5 \quad (9)$$

$$x_{inc} = \frac{5}{5} = 1.0, \quad y_{inc} = \frac{3}{5} = 0.6 \quad (10)$$

La Tabla 1 muestra las coordenadas generadas en cada iteración y el píxel activado tras aplicar el redondeo.

Table 1: Ejemplo de ejecución DDA.

Iteración	x	y	Píxel (x_{pixel}, y_{pixel})
0	0.0	0.0	(0,0)
1	1.0	0.6	(1,1)
2	2.0	1.2	(2,1)
3	3.0	1.8	(3,2)
4	4.0	2.4	(4,2)
5	5.0	3.0	(5,3)

6 Resultados

La implementación del algoritmo DDA fue evaluada mediante una aplicación gráfica interactiva desarrollada en Python utilizando el framework Flask. La aplicación permite al usuario seleccionar los puntos extremos del segmento mediante clics sobre el lienzo, configurar el color de la línea a través de controles para los canales RGB y modificar el tamaño de los píxeles con el fin de facilitar la visualización del proceso de rasterización.

La Figura 5 muestra el resultado obtenido al trazar un segmento diagonal sobre un fondo negro utilizando un color rojo puro ($R = 255$, $G = 0$, $B = 0$) y un tamaño de píxel igual a 5. Se observa que la línea generada está compuesta por una secuencia continua de píxeles que conecta correctamente los puntos seleccionados, sin presentar discontinuidades visibles.

Asimismo, la distribución de los píxeles sigue de manera adecuada la trayectoria de la recta ideal definida por los puntos extremos. Este comportamiento confirma que los incrementos calculados mediante las ecuaciones (6) y (7), junto con el proceso de redondeo descrito

en la ecuación (8), permiten obtener una representación discreta precisa del segmento de línea.

La visualización obtenida evidencia que el algoritmo genera una aproximación adecuada de la recta geométrica original, manteniendo la continuidad del trazado y una distribución uniforme de los píxeles a lo largo del segmento. Los resultados experimentales confirman, por tanto, el correcto funcionamiento del algoritmo DDA para el trazado de líneas en sistemas gráficos rasterizados.

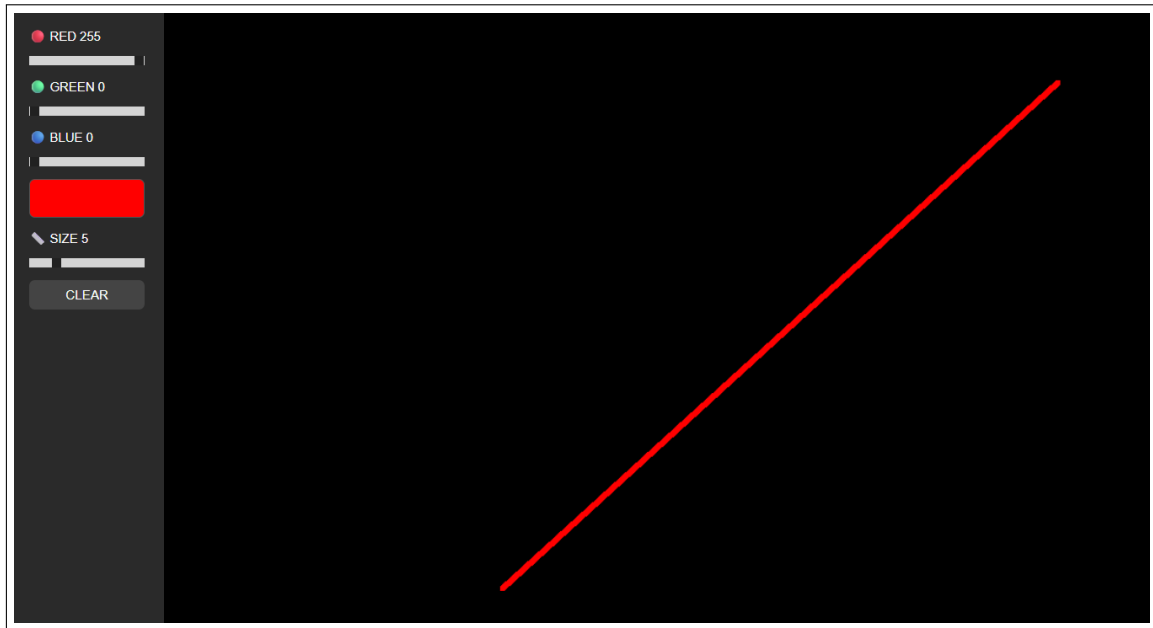


Figure 5: Resultado del trazado de una línea mediante el algoritmo DDA.

7 Conclusiones

El presente trabajo permitió analizar, formular e implementar el algoritmo DDA como una solución al problema de rasterización de líneas en sistemas gráficos discretos. A partir del estudio teórico y de los resultados experimentales obtenidos, se pueden establecer las siguientes conclusiones.

En primer lugar, el algoritmo DDA constituye una solución rápida y de fácil implementación para el trazado de líneas sobre una cuadrícula de píxeles. Su funcionamiento se basa en un único ciclo iterativo que calcula incrementos constantes en las coordenadas, evitando la evaluación repetitiva de la ecuación explícita de la recta. Esta simplicidad fue precisamente la razón de su elección para el presente trabajo, ya que permitió centrar el esfuerzo en comprender la formulación matemática que sustenta el algoritmo más que en la complejidad de su implementación.

En segundo lugar, aunque el algoritmo es eficiente en términos de complejidad temporal $O(n)$, su principal limitación radica en el uso de aritmética de punto flotante y la operación de redondeo, que introducen márgenes de error acumulativos conforme aumenta el número de iteraciones. Esta limitación es justamente la motivación que da origen al algoritmo de Bresenham, que resuelve el problema empleando únicamente aritmética entera.

Finalmente, la validación mediante Flask resultó una forma visualmente atractiva de ver-

ificar el funcionamiento del algoritmo. La incorporación de controles para modificar el color y el tamaño del píxel permite al usuario experimentar directamente sobre el lienzo, lo que facilita la comprensión del proceso de rasterización de manera interactiva.

Referencias

- Euresys. (s.f.). *Image coordinate systems*. Open eVision Documentation. https://documentation.euresys.com/Products/OPEN_EVISION/OPEN_EVISION_2_16/en-us/Content/03_Using/1_Starting_Up/3_Manipulating/Image_Coordinate_Systems.htm
- Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (1996). *Computer graphics: Principles and practice* (2nd ed.). Addison-Wesley.
- HandWiki. (2024). *Pixel structure in digital images*. <https://handwiki.org>
- Hearn, D., & Baker, M. P. (2014). *Computer graphics with OpenGL* (4th ed.). Pearson.
- Medium. (s.f.). *Line rasterization*. <https://medium.com>
- Ninamango Arroyo, B. R. (2026). *Algorithm DDA* [Repositorio de código]. GitHub. <https://github.com/Bryannsss140101/algorithm-DDA>
- Python Software Foundation. (2025). *Python documentation*. <https://docs.python.org/3/>
- Python Software Foundation. (2025). *Tkinter — Python interface to Tcl/Tk*. <https://docs.python.org/3/library/tkinter.html>
- Shirley, P., & Marschner, S. (2009). *Fundamentals of computer graphics* (3rd ed.). A K Peters/CRC Press.